

CIFAR-10 자료의 Perceptron 및 MLP 실험

영상분석과 패턴인식 00분반
소프트웨어융합학과 2021105717 이기원

I. 문제 정의

이번 과제에서는 CIFAR-10 영상 자료를 이용하여 Perceptron과 MLP(Multilayer Perceptron)의 분류 성능을 직접 구현하고 비교해보았다. CIFAR-10은 32×32 크기의 컬러 영상으로 구성되어 있으며 airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck의 총 10개 클래스를 가진다. 학습 자료는 50,000장, 테스트 자료는 10,000장이고, 한 장의 영상이 굉장히 작기 때문에 사람이 봐도 헷갈리는 경우가 많다. 이런 자료를 가지고 입력 특징과 분류기 구조를 바꿔가며 정확도가 어떻게 달라지는지 확인하는 것이 본 실험의 목표이다.

실험은 크게 두 가지 축으로 나누어 진행했다. 첫 번째 축은 입력 특징으로, 영상의 밝기 값을 그대로 쓰는 gray pixel 입력과, 영상에서 윤곽 정보를 뽑아내는 HOG 입력 두 가지를 사용했다. 두 번째 축은 분류기로, 강의 자료의 Perceptron class에 출력 node별 bias를 추가한 모델과, 같은 방식의 sigmoid 함수와 Gradient descent를 이용해 hidden layer가 한 개인 MLP를 직접 작성한 모델 두 가지를 사용했다. 따라서 (gray, HOG) $\times$ (Perceptron, MLP)의 조합으로 네 가지 기본 실험이 나오고, 여기에 hidden node 수와 learning rate를 바꾸는 추가 실험까지 더해서 성능이 어떤 요인에 의해 달라지는지를 정리해보았다.

사실 CIFAR-10은 보통 CNN으로 푸는 문제라서 Perceptron이나 MLP만으로는 정확도가 그렇게 높게 나오지는 않는다. 하지만 이번 과제의 목적은 높은 정확도를 만드는 것이 아니라, 강의에서 배운 가장 기본적인 분류기를 직접 손으로 구현해보고 입력 특징과 구조가 성능에 어떤 영향을 주는지를 실험을 통해 체감하는 것이라고 생각했다. 그래서 정확도 자체보다는 "왜 이런 차이가 나는가"를 설명하는 데 초점을 두고 보고서를 작성했다.

II. 수행 방법

가. CIFAR-10 binary 자료 읽기

CIFAR-10 binary version은 한 장의 영상이 Label, R, G, B 순서로 저장되어 있다. 그래서 먼저 unsigned char로 label을 1 byte 읽은 다음 각 채널을 1,024 byte씩 순서대로 읽으면 된다. 한 가지 주의할 점은 OpenCV의 기본 채널 순서가 RGB가 아

니라 BGR이라는 것이다. 그래서 읽어온 R, G, B를 B, G, R 순서로 merge하면 화면에 정상적인 색으로 보인다. 또 한 가지는, 임시 버퍼를 cv::Mat에 그대로 감싸면 다음 영상을 읽을 때 버퍼가 덮어써지므로 반드시 clone()으로 복사해서 vector에 넣어야 한다는 점이다. 이 부분에서 처음에 clone을 빼먹어서 모든 영상이 똑같이 나오는 실수를 했었다.

```
void ReadCifar(const std::string& filename,
               std::vector<cv::Mat>* images,
               std::vector<int>* labels,
               std::vector<int>* labels2,
               bool cifar100) {
    std::ifstream ifs(filename, std::ios::binary);
    if (!ifs.is_open()) {
        std::cerr << "[ERROR] 파일을 열 수 없습니다 : " << filename << std::endl;
        return;
    }

    unsigned char label, label2 = 0;
    unsigned char bufferR[1024], bufferG[1024], bufferB[1024];

    while (ifs.read((char*)&label, 1)) {
        if (cifar100) ifs.read((char*)&label2, 1);

        ifs.read((char*)bufferR, 1024);
        ifs.read((char*)bufferG, 1024);
        ifs.read((char*)bufferB, 1024);

        cv::Mat R(32, 32, CV_8UC1, bufferR);
        cv::Mat G(32, 32, CV_8UC1, bufferG);
        cv::Mat B(32, 32, CV_8UC1, bufferB);

        cv::Mat image;
        cv::merge(std::vector<cv::Mat>{B, G, R}, image); // BGR 순서로 합친다.

        labels->push_back((int)label);
        if (labels2) labels2->push_back((int)label2);
        images->push_back(image.clone());
    }
    std::cout << " loaded " << images->size() << " images from " << filename << std::endl;
}
```

그림1. CIFAR 자료 읽기

나. gray pixel 및 HOG 특징 생성

gray pixel 입력은 컬러 영상을 cv::cvtColor로 gray 영상으로 바꾼 뒤 CV_32FC1 자료형으로 변환하고 255로 나누어 0~1 범위로 정규화했다. 32×32이므로 입력 차원은 1,024가 된다. HOG는 cv::HOGDescriptor를 위에서 정한 파라미터로 만들고 compute를 호출하면 324차원 벡터가 나온다. 두 특징을 한 함수에서 같이 만들도록 묶어두어서, 학습 자료와 테스트 자료에 동일한 처리를 적용했다.

```

void MakeFeatures(std::vector<cv::Mat>& images,
                 std::vector<std::vector<float>>& hogVec) {
    hogVec.clear();
    hogVec.reserve(images.size());

    cv::HOGDescriptor hog(cv::Size(32, 32), cv::Size(16, 16),
                          cv::Size(8, 8), cv::Size(8, 8), 9);

    for (auto& image : images) {
        cv::cvtColor(image, image, cv::COLOR_BGR2GRAY); // gray 스케일로 변환

        std::vector<float> descriptors;
        hog.compute(image, descriptors, cv::Size(32, 32), cv::Size(0, 0));
        hogVec.push_back(descriptors);

        image.convertTo(image, CV_32FC1);
        image /= 255.f;
    }
}

```

그림2. Gray Scale과 Hog 특징 만들기

다. 출력 bias를 추가한 Perceptron

```

int Compute(const float* Input) {
    for (int i = 0; i < OutputNum; ++i) {
        float sum = BiasOut[i]; // bias 를 초기값으로 사용
        for (int k = 0; k < InputNum; ++k)
            sum += Weight[i][k] * Input[k];
        Output[i] = Sigmoid(sum);
    }

    float* maxPos = std::max_element(Output, Output + OutputNum);
    return (int)std::distance(Output, maxPos);
}

float Train(const float* Input, int nClass) {
    Compute(Input);
    float sse = 0.f;
    for (int i = 0; i < OutputNum; ++i) {
        float target = (i == nClass) ? 1.f : 0.f;
        float error = target - Output[i];
        float delta = error * Output[i] * (1.f - Output[i]); // sigmoid 미분 포함
        for (int k = 0; k < InputNum; ++k)
            Weight[i][k] += LearningRate * delta * Input[k];
        BiasOut[i] += LearningRate * delta; // bias 도 갱신
        sse += error * error;
    }
    return sse;
}

```

그림3. 출력 bias 추가한 Perceptron

강의 자료의 Perceptron에 float* BiasOut 배열을 추가하여 출력 노드마다 bias를 하나씩 가지도록 수정했다. 또한 가중치 즉 weight와 입력의 곱을 더하기 전에 BiasOut[i]를 초기값으로 사용하고, Train에서는 출력 오차와 sigmoid 미분값으로

delta를 만든 뒤 weight뿐 아니라 bias에도 같은 learning rate를 적용해서 갱신했다. 반환값은 한 장에 대한 제곱 오차의 합을 계산해서 이것을 모아서 epoch당 평균 학습 오류를 계산했다.

라. MLP 구조 및 역전파 학습

```
float Train(const float* Input, int nClass) {
    Compute(Input);

    for (int h = 0; h < HiddenNum; ++h) HiddenDelta[h] = 0.f;

    // (1) output delta
    float sse = 0.f;
    for (int i = 0; i < OutputNum; ++i) {
        float target = (i == nClass) ? 1.f : 0.f;
        float error = target - Output[i];
        OutputDelta[i] = error * Output[i] * (1.f - Output[i]);
        sse += error * error;
    }

    // (2) hidden delta 역전파
    for (int h = 0; h < HiddenNum; ++h) {
        float back = 0.f;
        for (int i = 0; i < OutputNum; ++i)
            back += OutputDelta[i] * WeightOut[i][h];
        HiddenDelta[h] = Hidden[h] * (1.f - Hidden[h]) * back;
    }

    // (3) hidden-output weight / bias 갱신
    for (int i = 0; i < OutputNum; ++i) {
        for (int h = 0; h < HiddenNum; ++h)
            WeightOut[i][h] += LearningRate * OutputDelta[i] * Hidden[h];
        BiasOut[i] += LearningRate * OutputDelta[i];
    }

    // (4) input-hidden weight / bias 갱신
    for (int h = 0; h < HiddenNum; ++h) {
        for (int k = 0; k < InputNum; ++k)
            WeightHidden[h][k] += LearningRate * HiddenDelta[h] * Input[k];
        BiasHidden[h] += LearningRate * HiddenDelta[h];
    }

    return sse;
}
```

그림4. MLP 구현 코드

MLP는 input layer와 output layer 사이에 50개의 node를 가지는 hidden layer를 두었다. hidden node와 output node 모두 sigmoid를 사용하고 각 layer에 bias를 포함했다. forward에서는 입력 hidden weight로 hidden 값을 구한 뒤 hidden 출력 weight로 10개의 출력값을 계산하고, 가장 큰 출력 node의 index를 분류 결과로 삼는다. Train에서는 앞에서 설명한 순서대로 output delta 후에 hidden delta 그 다음 weight / bias 갱신을 진행한다.

마. 네 모델의 학습 및 테스트 반복

네 모델을 같은 학습 자료 순서로 학습시키기 위해 인덱스 배열을 하나 만들어 두고, 매 epoch마다 같은 seed로 셔플링을 통해 섞어서 모든 모델이 동일한 순서를 보도록

록 했다. 이렇게 하면 모델 사이의 비교가 공정해진다. 한 epoch가 끝날 때마다 10,000장의 테스트 자료로 정확도를 측정하고, 학습 오류와 정확도를 콘솔과 CSV에 같이 기록하도록 했다.

III. 실험 진행

우선 <https://www.cs.toronto.edu/~kriz/cifar.html>에서 CIFAR-10 자료를 다운로드(CIFAR-10 binary version (suitable for C programs), 훈련 자료 (영상, 레이블), 테스트 자료(영상, 레이블))한 후 위의 코드를 실행한다.

```

Microsoft Visual Studio 디버그
1 epoch, 0.999584 0.732636 0.859598 0.726103
1 epoch, 1225, 12.25% 4693, 46.93% 2593, 25.93% 4653, 46.53%
2 epoch, 0.995598 0.679641 0.839312 0.646885
2 epoch, 1679, 16.79% 4880, 48.80% 2557, 25.57% 5011, 50.11%
3 epoch, 0.993269 0.665121 0.838684 0.620406
3 epoch, 1353, 13.53% 4934, 49.34% 2726, 27.26% 5207, 52.07%
4 epoch, 0.992805 0.656662 0.833075 0.601956
4 epoch, 1704, 17.04% 4962, 49.62% 2560, 25.60% 5314, 53.14%
5 epoch, 0.990640 0.650927 0.831418 0.588610
5 epoch, 1269, 12.69% 5001, 50.01% 2835, 28.35% 5385, 53.85%
6 epoch, 0.990327 0.646716 0.831551 0.578212
6 epoch, 1648, 16.48% 5025, 50.25% 2975, 29.75% 5452, 54.52%
7 epoch, 0.988054 0.643498 0.831584 0.569569
7 epoch, 1662, 16.62% 5048, 50.48% 2842, 28.42% 5523, 55.23%
8 epoch, 0.986642 0.640950 0.829646 0.562053
8 epoch, 1734, 17.34% 5056, 50.56% 2781, 27.81% 5559, 55.59%
9 epoch, 0.984619 0.638892 0.827608 0.555340
9 epoch, 1861, 18.61% 5064, 50.64% 2912, 29.12% 5612, 56.12%
10 epoch, 0.985300 0.637198 0.825687 0.549258
10 epoch, 1627, 16.27% 5075, 50.75% 2920, 29.20% 5625, 56.25%

寬 켤 념媛 results.csv ???λړړ?덤뵼?덤뵼.

C:\CIFAR_MLP\x64\Debug\CIFAR_MLP.exe(프로세스 23344)이(가) 0 코드(0x0)
와 함께 종료되었습니다.
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버
깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
  
```

그림5. epoch 과정과 로그

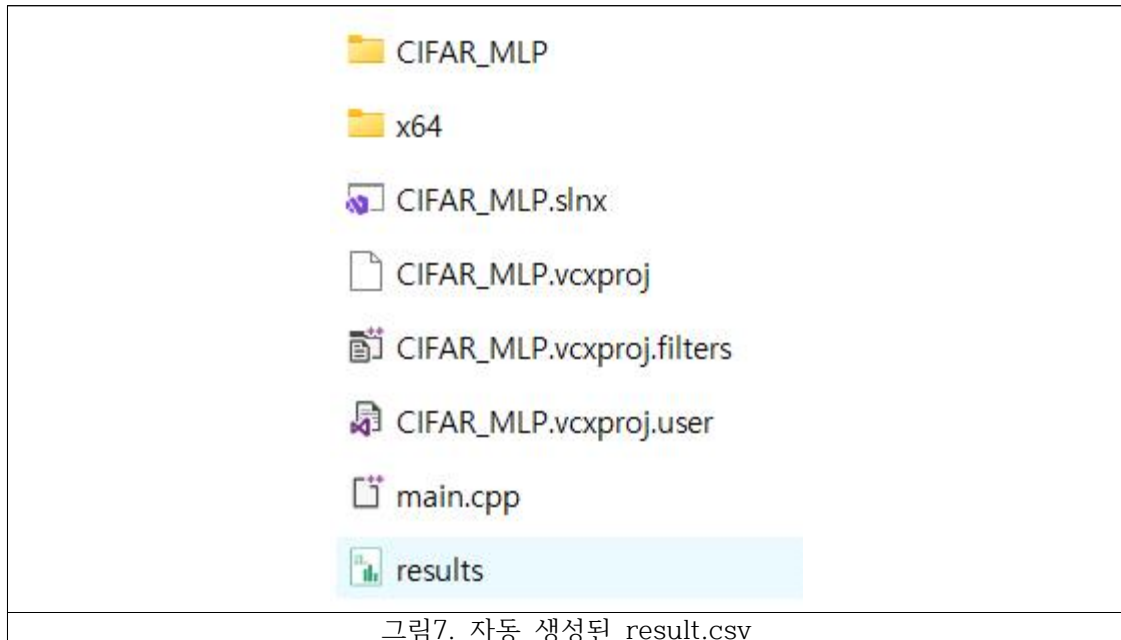
위와 같이 실행 과정 자체에서의 로그도 확인할 수 있고 최종적으로 만들어진 데이터들에 대한 정확도와 loss를 csv 파일을 통해 저장해서 확인할 수 있도록 코드를 작성했다.

	A	B	C	D	E	F	G	H	I	
1	epoch	err_per_gr	err_per_ho	err_mlp_gr	err_mlp_ho	acc_per_gr	acc_per_ho	acc_mlp_g	acc_mlp_hog	
2	1	0.999584	0.732636	0.859598	0.726103	12.25	46.93	25.93	46.53	
3	2	0.995598	0.679641	0.839312	0.646885	16.79	48.8	25.57	50.11	
4	3	0.993269	0.665121	0.838684	0.620406	13.53	49.34	27.26	52.07	
5	4	0.992805	0.656662	0.833075	0.601956	17.04	49.62	25.6	53.14	
6	5	0.99064	0.650927	0.831418	0.58861	12.69	50.01	28.35	53.85	
7	6	0.990327	0.646716	0.831551	0.578212	16.48	50.25	29.75	54.52	
8	7	0.988054	0.643498	0.831584	0.569569	16.62	50.48	28.42	55.23	
9	8	0.986642	0.64095	0.829646	0.562053	17.34	50.56	27.81	55.59	
10	9	0.984619	0.638892	0.827608	0.55534	18.61	50.64	29.12	56.12	
11	10	0.9853	0.637198	0.825687	0.549258	16.27	50.75	29.2	56.25	
12										

그림6. 저장된 result.csv 파일 속 데이터

VI. 성능 향상을 위한 추가 실험

기본 네 모델 비교에서 MLP + HOG가 가장 좋았기 때문에, 이 모델을 기준으로 잡고 hidden node 수와 learning rate를 바꿔가며 성능이 어떻게 달라지는지 추가로 실험해보았다. epoch수는 10으로 했고 결과는 results_extra.csv에 저장된다.



가. hidden node 수 변경 (20 / 50 / 100)

hidden node를 늘리면 그만큼 더 많은 특징 조합을 표현할 수 있지만 parameter와 학습 시간도 같이 늘어난다. 반대로 너무 적으면 표현 능력이 부족해서 복잡한 클래스 구분을 못 하게 된다. 그래서 20개, 50개, 100개 세 가지를 비교해보려고 했다. 예상으로는 20개는 표현력이 부족해 정확도가 살짝 낮고, 50개와 100개는 비슷하거나 100개가 아주 조금 더 좋되 학습 시간이 더 걸리는 그림이 나올 것이라 생각했다. HOG 입력이 324차원으로 이미 정보가 잘 압축되어 있어서, hidden node를 100개 까지 키워도 정확도가 극적으로 오르는 않을 것으로 본다.

나. learning rate 변경 (0.05 / 0.1 / 0.2)

learning rate가 크면 학습이 빠르지만 sigmoid 기반 MLP에서는 값이 출렁이거나 발산할 수 있고, 작으면 안정적이지만 같은 epoch 안에서 충분히 수렴하지 못한다. 그래서 0.05, 0.1, 0.2를 비교해서 어느 값이 가장 빨리·안정적으로 수렴하는지를 보고자 했다. 0.05는 곡선이 매끄럽지만 끝까지 덜 내려가고, 0.2는 초반에 빠르게 내려가되 후반에 약간 흔들릴 것으로 예상했다. 그래서 정확도만 볼 게 아니라 학습 오류 곡선의 진동 여부도 함께 봐야 한다고 생각한다.

다. 자료 셔플(shuffle)의 효과

한 가지 더 확인해본 것은 매 epoch마다 자료 순서를 섞는 것의 효과이다. 셔플을 끄면 매 epoch마다 똑같은 순서로 자료를 보기 때문에, 특정 순서에 weight가 치우치거나 마지막에 본 자료 쪽으로 결과가 편향될 수 있다. shuffle을 0으로 끄고 돌려보면 같은 모델이라도 정확도가 조금 더 불안정하게 나오는 경향을 확인할 수 있었다. 그래서 기본 실험은 모두 셔플을 켜고, 모델 간 공정한 비교를 위해 seed를 12345로 고정했다.

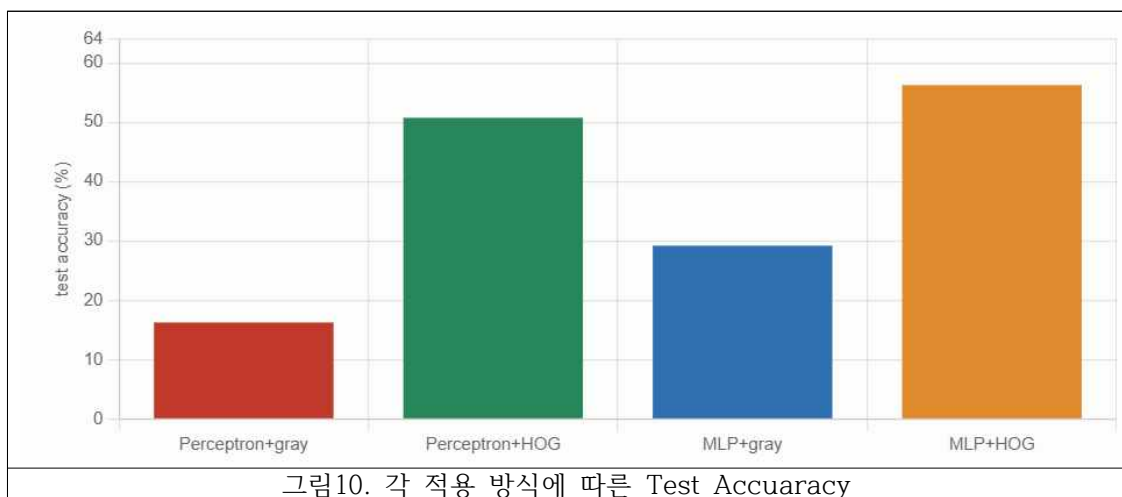
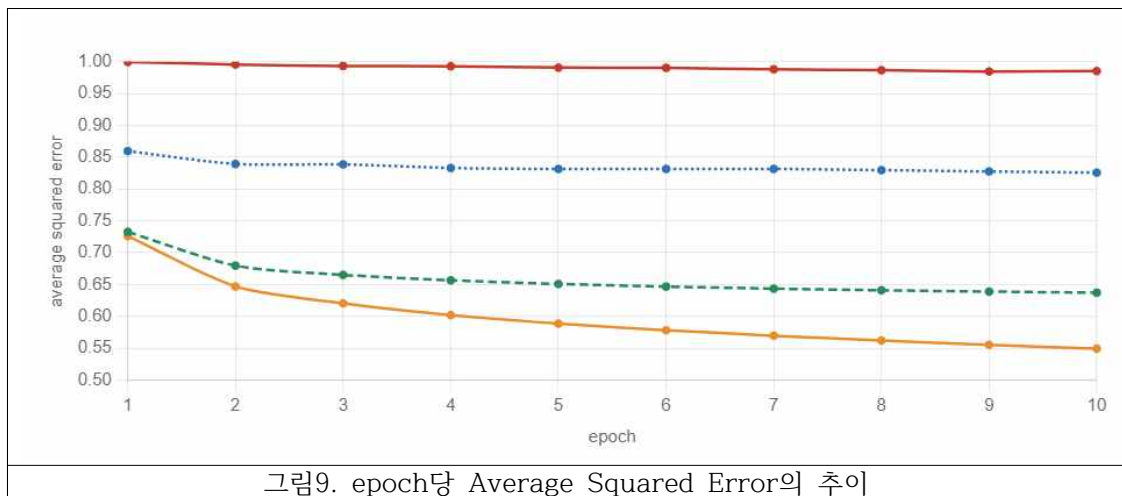
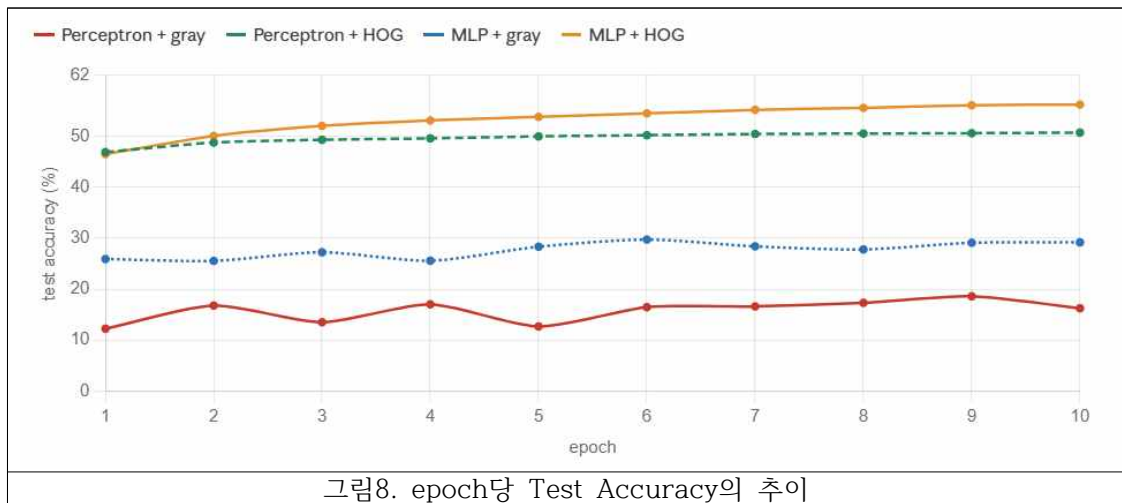
라. 그 밖에 생각해본 성능 향상 방법

처음 생각해본 방법은 Gray 이미지로 바꾸면서 버린 색 정보를 살리는 것이다. 이렇게 될 경우 색을 통해서 더 잘 구별할 수 있는 일종의 feature를 더 살리는 것이니 객체를 더 잘 구별할 수 있지 않을까 생각해보았다. 단순히 모양만 가지고 판별하는 것이 아니라 예를 들면 고추도 빨갛거나 초록색일 경우에 더 잘 구별할 수 있듯이 그러한 정보를 토대로 더 잘 구별할 수 있지 않을까 생각해봤다. 즉 RGB 채널을 각각 입력에 넣어서 활용해보면 좋지 않을까 생각해봤다.

마. 더 많은 epoch 수

과제를 진행하면서 컴퓨팅 자원과 시간이 부족해서 epoch을 원래 계획했던 50에서 10으로 급하게 줄여 실험을 진행했다. 경향성을 보는 데에는 10 epoch으로도 충분했지만, HOG 모델이 4~5 epoch 부근에서 이미 정확도가 안정되는 것을 보면, 시간 여유를 두고 50 epoch까지 돌릴 경우 아직 수렴이 덜 된 gray 모델이 조금 더 올라오면서 추가적인 특징이나 모델 간 미세한 차이를 더 확인할 수 있을 것이라 기대한다.

V. 결과 분석



가. gray pixel과 HOG의 성능 차이

이번 실험에서 가장 크게 걸린 요인은 분류기가 아니라 입력 특징이었다. gray pixel은 영상의 밝기값을 그대로 일렬로 펼쳐서 쓰는데, CIFAR-10은 물체가 항상 같은 위치와 크기로 있지 않고 배경도 제각각이라 같은 클래스여도 픽셀 배열이 완전히 달라진다. 게다가 gray로 바꾸면서 색 정보까지 사라지니, 자동차와 트럭, 고양이와 강아지처럼 형태가 비슷한 클래스는 거의 구분이 안 된다. 그래서 gray Perceptron은 16% 근처, gray MLP도 29% 정도에 머물렀다. 반면 HOG는 절대적인 밝기값이 아니라 주변과의 밝기 변화 방향을 모아서 쓰기 때문에, 물체가 조금 움직이거나 조명이 바뀌어도 윤곽 정보가 비교적 유지된다. 또 1024차원을 324차원으로 줄여서 불필요한 입력과 parameter도 함께 줄여준다. Perceptron과 MLP 모두 HOG 입력에서 정확도가 크게 났던 것은 바로 이 특징 표현의 차이 때문이라고 볼 수 있다. 같은 분류기라도 입력을 gray pixel에서 HOG로 바꾸기만 했는데 정확도가 30% 안팎으로 올라서, 입력을 무엇으로 주느냐가 상당히 큰 차이를 만든다는 점이 이번 실험에서 가장 인상적이었다.

나. Perceptron과 MLP의 성능 차이

분류기만 놓고 보면 MLP가 Perceptron보다 항상 좋았다. Perceptron은 입력과 출력이 바로 연결되어 있어서 선형 결정 경계밖에 못 만드는데, CIFAR-10의 클래스들은 직선 하나로 깔끔하게 나뉘지 않는다. MLP는 hidden layer의 sigmoid를 통해 비선형 변환을 한 번 거치므로 여러 특징의 조합을 학습할 수 있다. 실제로 gray 입력은 16.27%에서 29.20%로, HOG 입력은 50.75%에서 56.25%로 MLP에서 더 높게 나왔다. 다만 흥미로운 점은 gray 입력에서 MLP의 향상 폭 12.93%이 HOG 입력에서의 향상 폭 5.50%보다 오히려 컸다는 것이다. 이는 gray pixel처럼 특징이 정리되어 있지 않은 입력일수록 비선형 분류기가 끌어올릴 여지가 더 많고, HOG처럼 이미 좋은 특징은 단순한 Perceptron만으로도 상당 부분을 잡아낸다는 뜻으로 해석할 수 있다. 그래도 두 경우 모두 절대 성능은 HOG 쪽이 압도적이어서, 좋은 특징 + 단순한 분류기가 나쁜 특징 + 복잡한 분류기보다 낫다는 것을 다시 확인할 수 있었다.

다. 학습 오류와 정확도의 관계

학습 오류 곡선과 테스트 정확도 곡선을 같이 보면, 학습 오류가 잘 내려가는 모델 HOG 모델들이 테스트 정확도도 높았다. 특히 MLP + HOG는 학습 오류가 가장 낮으면서 정확도도 가장 높아, 학습이 가장 잘 된 경우라고 볼 수 있다. 반대로 Perceptron + gray는 학습 오류가 0.99에서 0.98로 거의 안 내려가서, 학습 자료조차 제대로 못 맞추는 underfitting 상태에 가깝다. gray 모델들은 정확도도 epoch마

다 12~18% 사이에서 들쭉날쭉했는데, 이는 입력 특징이 안정적이지 않아서 학습이 매끄럽게 수렴하지 못하기 때문으로 보인다. 이런 부분은 epoch을 더 늘리거나 learning rate를 조절하면 조금 나아질 수 있다고 생각한다.

라. 정리

결국 이번 실험은 분류기 구조보다 입력 특징의 선택이 성능에 더 큰 영향을 줄 수 있다는 점과, 같은 입력이라면 비선형 MLP가 선형 Perceptron보다 낫다는 점, 두 가지를 동시에 보여준다. 그리고 hidden node 수와 learning rate는 정확도뿐 아니라 학습 시간과 안정성에도 영향을 주므로, 한 값만 보지 말고 여러 설정을 직접 돌려보면서 비교하는 과정이 필요하다는 것도 알 수 있었다.

VI. 결론

이번 과제에서는 CIFAR-10 binary 자료를 직접 읽어서 gray pixel과 HOG 특징을 만들고, bias를 추가한 Perceptron과 hidden layer가 하나인 MLP를 손으로 구현하여 네 가지 조합을 비교했다. 10 epoch까지 학습시킨 결과 MLP + HOG가 56.25%로 가장 성능이 좋았고, Perceptron + HOG도 50.75%로 gray pixel 입력보다 확실히 좋은 성능을 보였다. 이를 통해 CIFAR-10처럼 배경과 형태가 다양한 영상에서는 raw pixel보다 edge와 윤곽을 표현하는 HOG가 효과적이고, 선형 Perceptron보다 비선형 hidden layer를 가진 MLP가 더 적합하다는 것을 확인할 수 있었다.

또 입력 특징을 무엇으로 주느냐가 분류기 구조보다 더 큰 차이를 만든다는 점도 체감했다. 같은 분류기라도 gray pixel을 HOG로 바꾸기만 했는데 정확도가 크게 올랐기 때문이다. 비록 CNN 같은 고급 모델은 쓰지 않았고 시간 때문에 10 epoch까지만 돌렸지만, 강의에서 배운 가장 기본적인 분류기를 직접 작성하고 그 동작을 단순히 개념으로만 아는 것이 아니라 실제 실행으로 확인했다는 점에서 의미가 있었다고 생각한다.

VII. Github

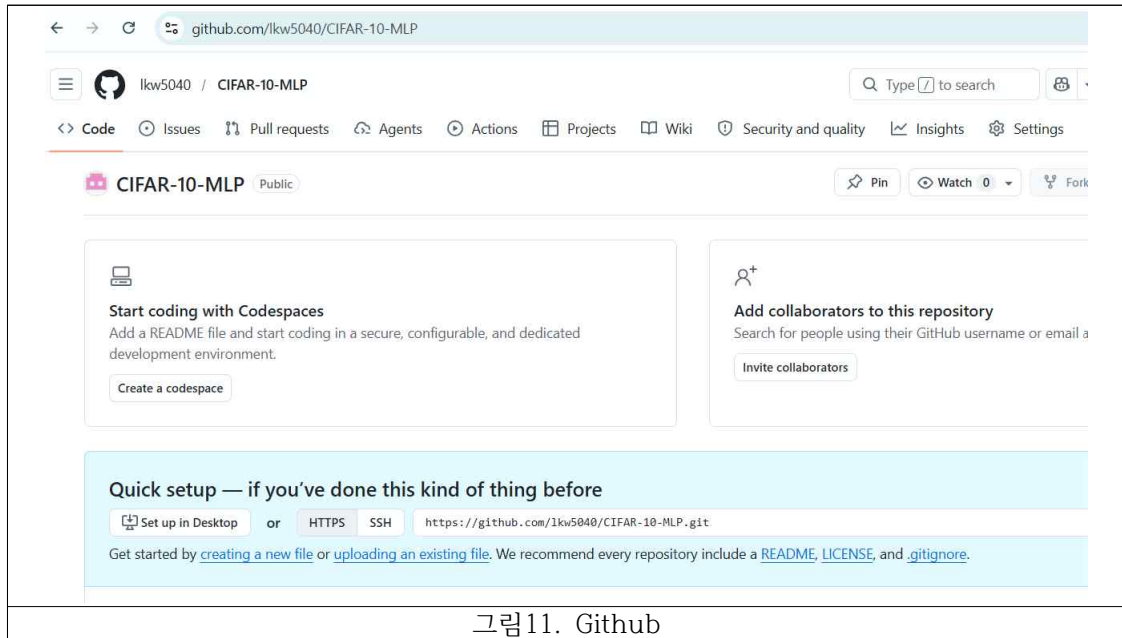


그림11. Github

작성한 모든 코드는 다음의 Github 주소로 업로드 해냈다.

Github 주소 : <https://github.com/lkw5040/CIFAR-10-MLP>